

ORM and Description Logic

Dr. Mustafa Jarrar

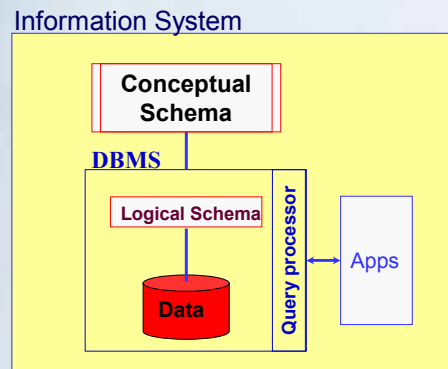
mjarrar@vub.ac.be

STARLab, Vrije Universiteit Brussel,

Outline

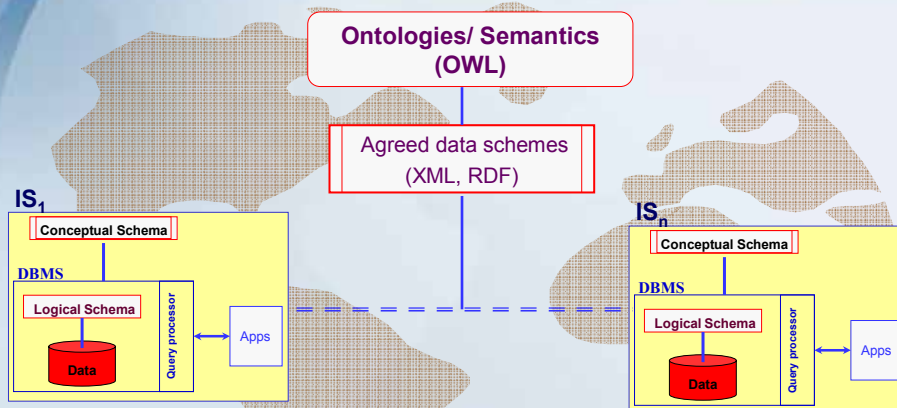
- Introduction (Why this tutorial)
- Part I: Introduction to Description logics
- Part II: Mapping ORM into Description logic
- Something to think about ☺

An Information System



- Each Information System is made for one organization.
- Why do we need conceptual schemes? for designing Information systems at the conceptual level.

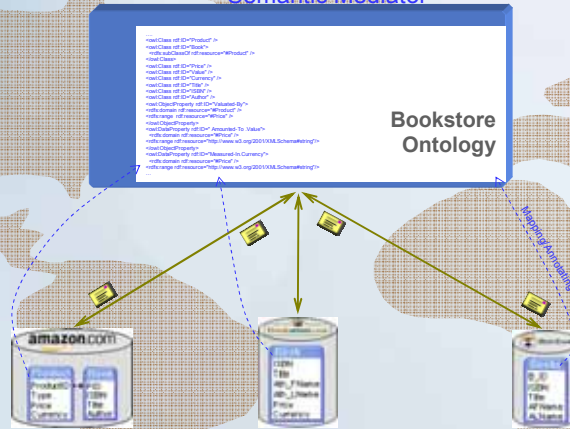
(Web) Information Systems



New needs:
Open data exchange, inter-organizational transactions, global queries...

(Web) Information Systems

Semantic Mediator



5

OWL

- The Web Ontology Language (W3C Standard)
- Based on Description logic

```
....
<owl:Class rdf:ID="Product" />
<owl:Class rdf:ID="Book">
  <rdfs:subClassOf rdf:resource="#Product" />
</owl:Class>
<owl:Class rdf:ID="Price" />
<owl:Class rdf:ID="Value" />
<owl:Class rdf:ID="Currency" />
<owl:Class rdf:ID="Title" />
<owl:Class rdf:ID="ISBN" />
<owl:Class rdf:ID="Author" />
<owl:ObjectProperty rdf:ID="Valuated-By">
  <rdfs:domain rdf:resource="#Product" />
  <rdfs:range rdf:resource="#Price" />
</owl:ObjectProperty>
<owl:DataProperty rdf:ID="Amounted-To .Value">
  <rdfs:domain rdf:resource="#Price" />
  <rdfs:range rdf:resource="http://www.w3.org/2001/XMLSchema#string"/>
</owl:DataProperty>
...
```

➔ Can we use ORM to Model OWL ontologies?

6

A short Introduction to Description logics

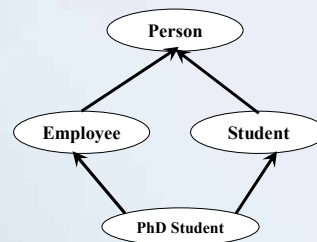
Part I

First Order Logic (FOL)

8

- FOL allows us to represent knowledge precisely (Syntax and Semantics).

$\forall x \text{ Employee}(x) \rightarrow \text{Person}(x)$
 $\forall x \text{ Student}(x) \rightarrow \text{Person}(x)$
 $\forall x \text{ PhDStudent}(x) \rightarrow \text{Student}(x)$
 $\forall x \text{ PhDStudent}(x) \rightarrow \text{Employee}(x)$

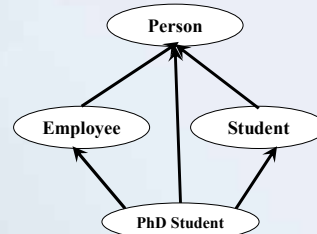


- However, representation alone is not enough.
- We also need to process this knowledge and make use of it, i.e. Logical inference = (Reasoning).

First Order Logic (FOL)

Reasoning:

- $\forall x \text{ Employee}(x) \rightarrow \text{Person}(x)$
- $\forall x \text{ Student}(x) \rightarrow \text{Person}(x)$
- $\forall x \text{ PhDStudent}(x) \rightarrow \text{Student}(x)$
- $\forall x \text{ PhDStudent}(x) \rightarrow \text{Employee}(x)$
- $\forall x \text{ PhDStudent}(x) \rightarrow \text{Person}(x)$

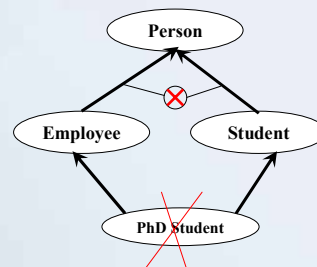


→ How to process the above *axioms* to know that an axiom can be derived from another axiom.

First Order Logic (FOL)

Reasoning:

- $\forall x \text{ Employee}(x) \rightarrow \text{Person}(x)$
- $\forall x \text{ Student}(x) \rightarrow \text{Person}(x)$
- $\forall x \text{ PhDStudent}(x) \rightarrow \text{Student}(x)$
- $\forall x \text{ PhDStudent}(x) \rightarrow \text{Employee}(x)$
- $\forall x \text{ Student}(x) \cap \text{Employee}(x) = \emptyset$



→ How to process the above *axioms* to know that an axiom can be derived from another axiom.

→ Find contradictions (satisfiability)

→ ...etc.

First Order Logic (FOL)

Reasoning FOL is far too complex (i.e. not decidable)

→ Here comes description logics.

Description Logics

- Description logics are a family of logics concerned with knowledge representation.
- A description logic is a **decidable** fragment of first-order logic, associated with a set of automatic **reasoning** procedures.
- The basic constructs for a description logic are the notion of a concept and the notion of a relationship.
- Complex concept and relationship expressions can be constructed from atomic concepts and relationships with suitable constructs between them.
- Example: $\text{HumanMother} \sqsubseteq \text{Female} \sqcap \exists \text{HasChild}.\text{Person}$

Description Logics

Most known description logics are :

FL^- The simplest and less expressive description logic.
 $C, D \rightarrow A \mid C \sqcap D \mid \forall R.C \mid \exists R$

AL A more practical and expressive description logic.
 $C, D \rightarrow A \mid \top \mid \perp \mid \neg A \mid C \sqcap D \mid \forall R.C \mid \exists R.\top$

$SHOIN$ The most famous description logic.
 The logic underlying OWL.

DLR_{idf} The most expressive description logic,
 Capable of representing most database constructs.

Description Logic Reasoners

FaCT++

Racer

Pellet

- They offer reasoning services for multiple TBoxes and ABoxes.
- They run as background reasoning engines.
- They understand DIG, which is a simple protocol (based on HTTP) along with an XML Schema.
- Example: $Student \sqsubseteq Person$

```
<implies>
  <catom name="Student"/>
  <catom name="Person"/>
</implies>
```

The **DRL** description logic

Concepts denoted by C and arbitrary relations denoted by R , can be built according to the following syntax respectively:

$$C ::= \top_1 \mid A \mid \neg C \mid C_1 \sqcap C_2 \mid (\leq k[i]R)$$

$$R ::= \top_n \mid P \mid (i/n : C) \mid \neg R \mid R_1 \sqcap R_2$$

where A is an atomic concept, P is an atomic relation, n denotes the arity of the relations P , R , R_1 and R_2 , i denotes a component of a relationship, and k denotes a non-negative integer.

The semantics of **DRL**:

$$\begin{array}{ll} \top_n^I \subseteq (\Delta_n)^I & \top_1^I = \Delta^I \\ P^I \subseteq \top_n^I & A^I \subseteq \Delta^I \\ (i/n : C)^I = \{t \in \top_n^I \mid t[i] \in C^I\} & (\neg C)^I = \Delta^I \setminus C^I \\ (\neg C)^I = \top_n^I \setminus R^I & (C_1 \sqcap C_2)^I = C_1^I \cap C_2^I \\ (R_1 \sqcap R_2)^I = R_1^I \cap R_2^I & (\leq k[i]R)^I = \{a \in \Delta^I \mid \#\{t \in R_1^I \mid t[i] = a\} \leq k\} \end{array}$$

Example

DRL Knowledge Base

DRL TBox	$Person \sqsubseteq \exists HasBirthDay.STRING$
	$Student \sqsubseteq Person$
	$Professor \sqsubseteq Person$
	$Professor \sqsubseteq \exists Teaches.Course$
	$Registration \sqsubseteq (1: Student) \sqcap (2: Course) \sqcap (3: Grade)$
DRL ABox	$Professor(Robert)$
	$Teaches(Robert, Database-I)$
	$Student(John)$
	$Registration(John, Database-I, 17)$

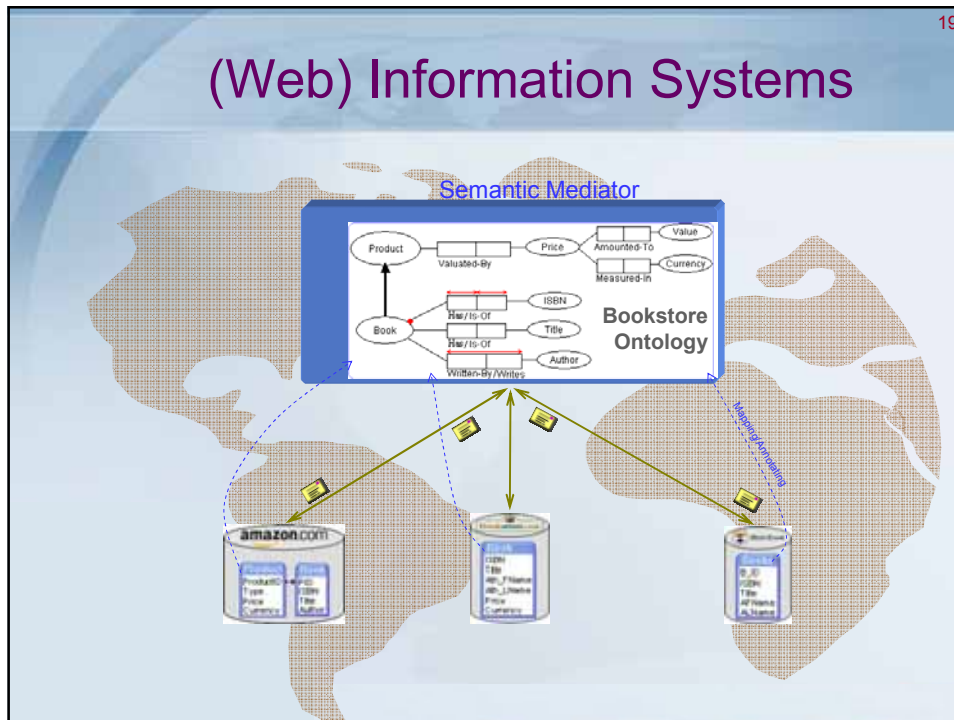
In Part II, we will see

- More examples
- How can we map ORM into Description logics
- How to reason about ORM schemes

Mapping ORM into Description Logic

Part II

(Web) Information Systems



Mapping ORM to Description Logic

Why do we need this mapping for?

- Us ORM as a graphical notation for Ontology/description logic languages.
(The DL benefit from ORM)

- Reasoning on ORM schemes automatically.

(The ORM benefit from DL)

Reasoning Services

Examples of reasoning services:

- Satisfiability

To know whether a concept can be populated or not (e.g. because of some axioms contradicting each other)

$\text{SAT}(C, \mathcal{T})$ iff there is a model $\mathcal{I}$ of $\mathcal{T}$ with $C' \neq \emptyset$

- Subsumption

To know whether a concept is subsuming another concept (e.g. to find unwanted or missing subsumptions)

$\text{SUBS}(C, D, \mathcal{T})$ iff $C' \subseteq D'$ for all model $\mathcal{I}$ of $\mathcal{T}$

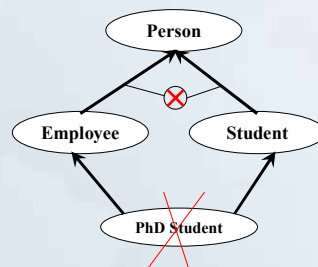
- Redundancies

To know whether two concepts are equal (e.g. to find out redundancies)

$\text{EQUIV}(C, D, \mathcal{T})$ iff $C' = D'$ for all model $\mathcal{I}$ of $\mathcal{T}$

Reasoning on ORM Schemes

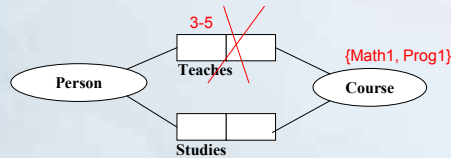
(Constraint Contradictions, Example 1)



- Contradiction**
- The exclusion constraint says there is no 'Person' who can be an 'Employee' and a 'Student' at the same time, i.e. the intersection of 'Employee' and 'Student' should be empty.
 - 'PhD Student' sub-type of both 'Employee' and 'Student', i.e. it is the intersection of both.
- The concept 'PhD Student' is not satisfiable, i.e. will never be populated, always empty.

Reasoning on ORM Schemes

(Constraint Contradictions, Example 2)

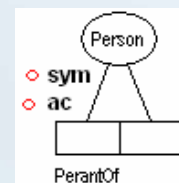
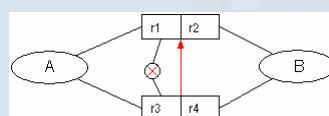
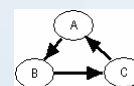
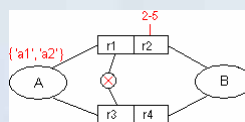
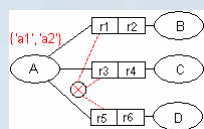
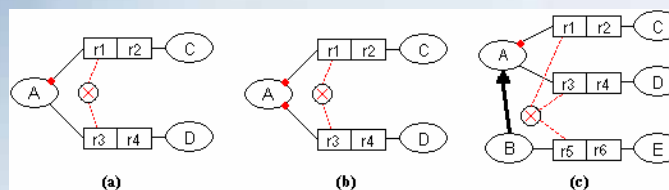


Contradiction

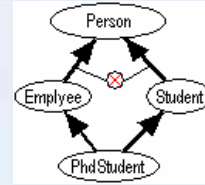
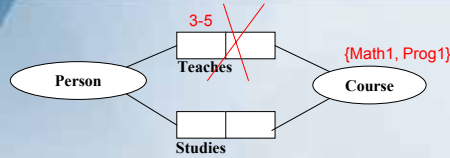
- The frequency constraint means that: Each 'Person' must teach at least 3 and most 5 different 'Courses'.
 - The value constraint means that: there are only two possible 'Courses', which are {'Math1', 'Prog1'}.
- The role 'Teaches' is not satisfiable, i.e. will never be populated, always empty.

Reasoning on ORM Schemes

(Constraint Contradictions, Other Examples)



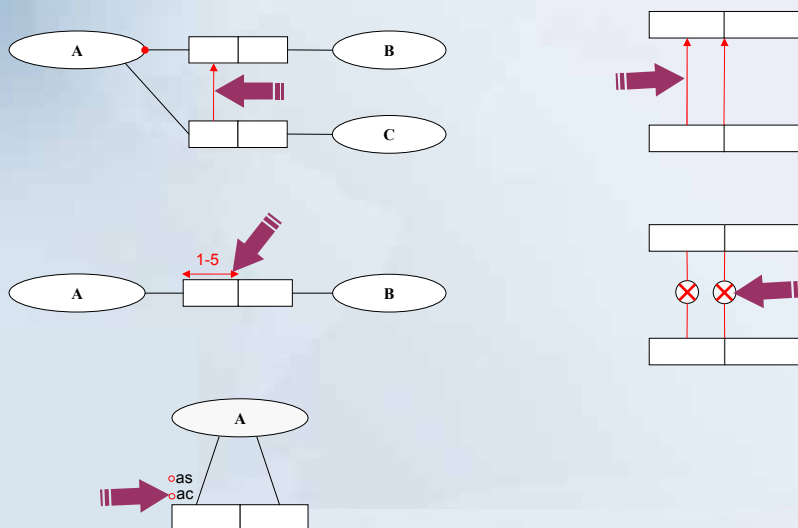
Reasoning on ORM Schemes



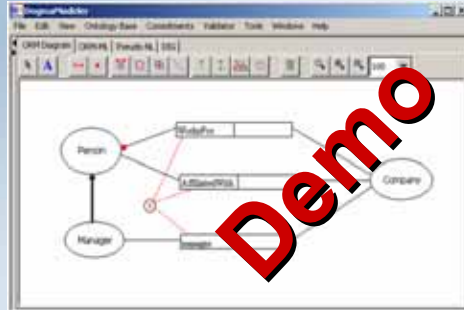
- Schema satisfiability: A schema is satisfiable if and only if there is at least one concept in the schema that can be populated. → **Weak satisfiability**
 - Concept satisfiability: A schema is satisfiable if and only if all concepts in the schema can be populated.
 - Role satisfiability: A schema is satisfiable if and only if all roles in the schema can be populated. → **Strong satisfiability**
- Concept satisfiability implies schema satisfiability.
→ Role satisfiability implies concept satisfiability.

Reasoning on ORM Schemes

(Constraint Implications, Examples)



DogmaModeler



- DogmaModeler is an ontology engineering tool
- It uses ORM as a graphical notation
- It uses Racer as a background reasoning engine

Other functionalities of DogmaModeler:

- Verbalization of ORM into 11 human languages.
- Modularization of auto composition of ORM schemes
-

Questions

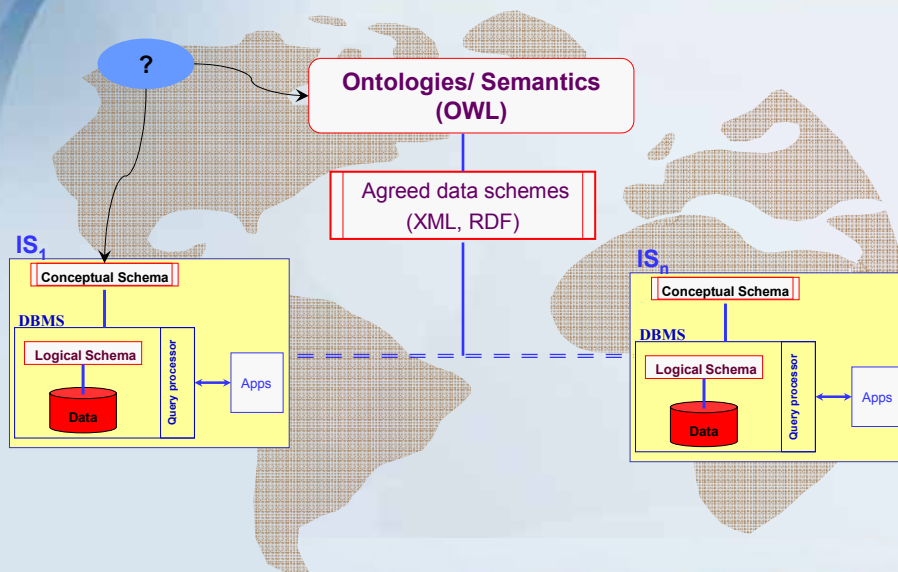


I have a question

Is there any difference between an ontology and a data schema?

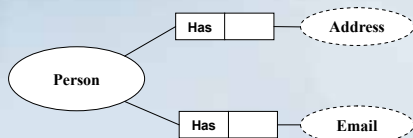
→ We don't have go in details now ...

(Web) Information Systems



Example

Is this an Ontology or a Data Schema?

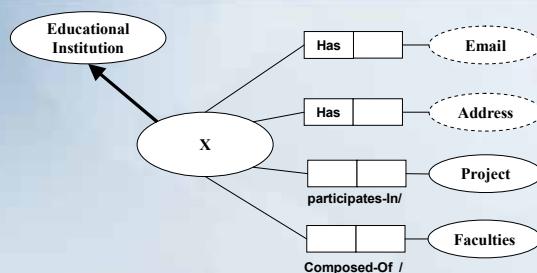


```

<owl:Class rdf:ID="Person" />
<owl:Class rdf:ID="Address" />
<owl:Class rdf:ID="email" />
<owl:DataProperty rdf:ID="Has-Address">
  <rdfs:domain rdf:resource="#Person" />
  <rdfs:range rdf:resource="www.w3.org/2001/XMLSchema#string"/>
</owl:ObjectProperty>
<owl:DataProperty rdf:ID="Has-Email">
  <rdfs:domain rdf:resource="#Person" />
  <rdfs:range rdf:resource="www.w3.org/2001/XMLSchema#string"/>
</owl:ObjectProperty>
  
```

What makes an ontology an ontology?

Example, What is X?



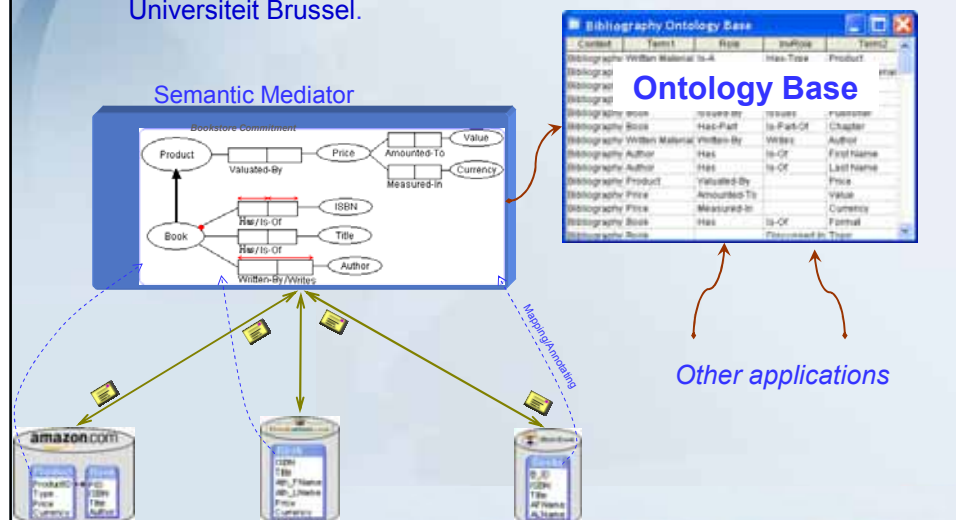
- Which of these characteristics are more distinguishing?
(*Intrinsic* verse *extrinsic* characteristics)

An ontology (in whatever language it is specified) should capture not only the extrinsic, but most of the intrinsic characteristics, while a data schema can have only the extrinsic.

The important thing is what we model, not how we model.

Dogma

An ontology engineering frame work, developed at STARLab, Vrije Universiteit Brussel.



To Read More

About Description logic

- Online Course by Enrico Franconi <http://www.inf.unibz.it/~franconi/dl/course/>

ORM-Description logic

- Mustafa Jarrar and Mohammed Eldammagh: Reasoning on ORM using Racer. Technical Report. STAR Lab, Vrije Universiteit Brussel, Belgium. August 2007.
- Mustafa Jarrar and Enrico Franconi: Formalizing ORM using the DLR description logic. (Coming soon)

Dogma / Ontology

- Mustafa Jarrar and Robert Meersman: The DOGMA Approach of Ontology Engineering. In Advances in Web Semantic. Volume 1, A state-of-the Art Semantic Web Advances in Web Semantics IFIP2.12. Chapter 3. Springer. 2007.
- Mustafa Jarrar: Towards methodological principles for ontology engineering. PhD Thesis. Vrije Universiteit Brussel. (May 2005)

→ <http://www.starlab.vub.ac.be/staff/mustafa/publications>

Thank You

Mustafa Jarrar
mjarrar@vub.ac.be
STARLab, Vrije Universiteit Brussel, Belgium
<http://www.starlab.vub.ac.be/staff/mustafa>